

HDOC Day-24

Generation of Increasing Patterns Using a Menu-Driven C Program

Aim

To write a menu-driven C program using a switch statement, functions, and nested loops to generate different increasing patterns based on the user's choice, without using arrays.

Algorithm

1. Start the program.
2. Display a menu containing the following pattern options: Increasing right-angled triangle, Increasing row-number triangle, Increasing natural-number pattern, Consecutive-number triangle, Increasing alphabet pattern, and Exit.
3. Accept the user's choice.
4. Use a switch statement to call the corresponding parameterless function.
5. Inside each function, accept the required number of rows n.
6. Use nested loops to generate the selected pattern.
7. For the alphabet pattern, accept values of n from 1 to 26.
8. Display an error message for an invalid menu choice.
9. Repeat the menu until the user selects Exit.
10. Stop the program.

C Program

```
#include <stdio.h>

void starTriangle();
void rowNumberTriangle();
void naturalNumberPattern();
void consecutiveNumberTriangle();
void alphabetPattern();

int main()
{
    int choice;

    do
    {
        printf("\n----- INCREASING PATTERNS MENU ----- \n");
        printf("1. Increasing Right-Angled Triangle\n");
        printf("2. Increasing Row-Number Triangle\n");
        printf("3. Increasing Natural-Number Pattern\n");
        printf("4. Consecutive-Number Triangle\n");
        printf("5. Increasing Alphabet Pattern\n");
        printf("6. Exit\n");

        printf("Enter your choice: ");
```

```

        scanf("%d", &choice);

        switch (choice)
        {
            case 1: starTriangle(); break;
            case 2: rowNumberTriangle(); break;
            case 3: naturalNumberPattern(); break;
            case 4: consecutiveNumberTriangle(); break;
            case 5: alphabetPattern(); break;
            case 6: printf("Exiting program...\n"); break;
            default: printf("Invalid choice! Please enter a number from 1 to 6.\n");
        }
    } while (choice != 6);

    return 0;
}

void starTriangle()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
}

void rowNumberTriangle()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++) printf("%d ", i);
        printf("\n");
    }
}

void naturalNumberPattern()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++) printf("%d ", j);
        printf("\n");
    }
}

void consecutiveNumberTriangle()
{
    int n, i, j, num = 1;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", num);
            num++;
        }
        printf("\n");
    }
}

```

```

void alphabetPattern()
{
    int n, i, j;
    printf("Enter number of rows (1-26): ");
    scanf("%d", &n);
    if (n < 1 || n > 26)
    {
        printf("Invalid number of rows!\n");
        return;
    }
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++) printf("%c ", 'A' + j - 1);
        printf("\n");
    }
}

```

Compilation/Execution Evidence

```

(kali@kali)-[~/Desktop/demo1]
$ gcc HDOC24.c

```

```

(kali@kali)-[~/Desktop/demo1]
$ ./a.out

—— INCREASING PATTERNS MENU ——
1. Increasing Right-Angled Triangle
2. Increasing Row-Number Triangle
3. Increasing Natural-Number Pattern
4. Consecutive-Number Triangle
5. Increasing Alphabet Pattern
6. Exit
Enter your choice: 1
Enter number of rows: 4
*
* *
* * *
* * * *

—— INCREASING PATTERNS MENU ——
1. Increasing Right-Angled Triangle
2. Increasing Row-Number Triangle
3. Increasing Natural-Number Pattern
4. Consecutive-Number Triangle
5. Increasing Alphabet Pattern
6. Exit
Enter your choice: 2
Enter number of rows: 4
1
2 2
3 3 3
4 4 4 4

```

—— INCREASING PATTERNS MENU ——

1. Increasing Right-Angled Triangle
2. Increasing Row-Number Triangle
3. Increasing Natural-Number Pattern
4. Consecutive-Number Triangle
5. Increasing Alphabet Pattern
6. Exit

Enter your choice: 3

Enter number of rows: 4

1 *Trash*

1 2

1 2 3

1 2 3 4

—— INCREASING PATTERNS MENU ——

1. Increasing Right-Angled Triangle
2. Increasing Row-Number Triangle
3. Increasing Natural-Number Pattern
4. Consecutive-Number Triangle
5. Increasing Alphabet Pattern
6. Exit

Enter your choice: 5

Enter number of rows (1-26): 4

A

A B

A B C

A B C D

—— INCREASING PATTERNS MENU ——

1. Increasing Right-Angled Triangle
2. Increasing Row-Number Triangle
3. Increasing Natural-Number Pattern
4. Consecutive-Number Triangle
5. Increasing Alphabet Pattern
6. Exit

Enter your choice: 5

Enter number of rows (1-26): 27

Invalid number of rows!

—— INCREASING PATTERNS MENU ——

1. Increasing Right-Angled Triangle
2. Increasing Row-Number Triangle
3. Increasing Natural-Number Pattern
4. Consecutive-Number Triangle
5. Increasing Alphabet Pattern
6. Exit

Enter your choice: 6

Exiting program ...

Test Case Table

Test Case	Choice	Input	Expected Output
1	1	n = 4	*, *, *, *, *, *, *, *
2	2	n = 4	1; 2 2; 3 3 3; 4 4 4 4
3	3	n = 4	1; 1 2; 1 2 3; 1 2 3 4
4	4	n = 4	1; 2 3; 4 5 6; 7 8 9 10
5	5	n = 4	A; A B; A B C; A B C D
6	5	n = 27	Invalid number of rows!
7	7	—	Invalid choice!
8	6	—	Program exits

Result

The menu-driven C program was successfully designed to generate the five required increasing patterns. The program uses a switch statement, parameterless functions, and nested loops without arrays, as specified in the assignment. It also handles invalid menu choices and validates the allowed range for the alphabet pattern.